Evaluating Neural Network Approaches for Mathematical Problem Classification, with Applications in Auto-Tagging for Education

Soraya Butler
MSDS 458: AI & Deep Learning
March 15, 2026

1.0 Abstract

Auto-tagging applications are widely used in education and e-learning to facilitate curriculum development, pedagogical alignment, content recommendations, and education management. This paper describes a series of experiments using neural networks for sequential data to classify the Kaggle dataset "KAChallenges Series 1: Classifying Math Problems," serving as a benchmark for model efficacy in a potential auto-tagging application. 11 exploratory data analysis experiments were conducted to determine parameters for text vectorization and text embeddings. Afterwards, 13 experiments on neural network architectures were conducted, using varying network topologies to measure trade-offs among accuracy, model fit, and training time. The results of this study demonstrate that using embeddings, rather than vectorization, to represent text generally increased model accuracy, with a fine-tuned BERT model achieving the highest accuracy. However, underrepresented labels are classified poorly across all models, suggesting that solutions to address data scarcity and class imbalance are required. Future experiments are recommended to improve model accuracy and generalizability.

2.0 Introduction

This report describes a series of experiments using neural networks for math text topic classification, to help address a broader management problem in auto-tagger development for education. A series of 13 experiments was conducted to determine the neural network architecture that best balances accuracy and model fit for the Kaggle dataset "KAChallenges Series 1: Classifying Math Problems" classification task. Exploratory data analysis is conducted on the math text data to determine hyperparameters for vectorization and embedding. Various network topologies are applied to the classification problem, including Long Short-Term

Memory (LSTM), Transformer, and Bidirectional Encoder Representations from Transformers (BERT). Benchmarked models are further evaluated for use in a production-ready auto-tagger for educational purposes. The experiments were performed in Python on a Google Colab Pro notebook using TensorFlow Keras. Relevant figures for the study are in Appendix A, and the source code and model outputs are provided in Appendix B.

3.0 Literature Review

Auto-tagging in education involves automatically tagging educational content, such as questions, textbooks, lecture notes, presentations, lessons, and learning outcomes. This tool enables large volumes of educational content to be classified automatically, for use in pedagogical alignment, content recommenders, and question management (Grévisse, Braun and Batista da Costa 2025; Wang, Ding and Chen 2022). Auto-tagging also reduces teachers' labor in curriculum development and minimizes bias in tagging decisions. Automatic classification for math problems is especially interesting, since mathematical text contains symbols, logic, and hierarchical knowledge points.

Several similar studies have examined the problem of educational content classification for auto-tagging. In their 2023 study, Lee et al. investigated how to mitigate data scarcity in educational text classification for auto-tagging, a problem exacerbated by a large label space in which rare labels suffer from worse classification performance. To address this, the authors of the study viewed the problem through a retrieval lens, where a language model understands the input and retrieves the relevant tag. The intention is that, even for rare or unseen tags, the model can capture the semantic relationship between the input and the tag and classify appropriately. Cross-encoding is used to augment the data with input-tag relationship information, and a

bi-encoder is used for retrieval. The authors found that their model improves over the state-of-the-art tagging model at the time, TagRec++. While similar in objective to the study by Lee et al., this paper differs in its focus on math text classification rather than classification across multiple subjects. This paper is further distinguished by an iterative approach that isolates the effect of individual hyperparameters on model performance.

A more comparable study to this one was conducted in 2022 by Wang et al., focusing on K-12 math text topic classification using an LSTM-based architecture. K-12 math text questions were classified using a novel model called Label Attention Basic Label Smoothing (LABS). The model uses TangentCFT to parse formulas, bidirectional long short-term memory (BiLSTM) for encoding, label-semantic attention to update weights, and multi-label smoothing for regularization. The authors iteratively tested their model with increasing complexity, starting with only the “Basic” vectorization with BiLSTM, then layering in label-semantic attention, smoothing, and so on. The authors found that the most complex iteration had the greatest accuracy, recall, and F1 score. While this paper and the one by Wang et al. both focus on math text classification using iterative experimentation with neural network architectures, this paper experiments with different neural network approaches, including not only LSTM but also BERT and MathBERT.

4.0 Methods

Keras, TensorFlow, and PyTorch, running in a Google Colab Pro environment with an L4 GPU runtime, are used to develop and evaluate 13 neural network models. Several other Python libraries were required for experimentation, including Pandas, NumPy, scikit-learn, seaborn, and Matplotlib. Although network topologies varied, some parameters were kept constant across

experiments. All experiments use the ReLU activation function, softmax output activation, the Adam optimizer, and sparse categorical cross-entropy loss. When early stopping is not used, the epoch count is set to 5. Otherwise, the epoch count can reach 200. All models output the same set of evaluation metrics and visualizations: test accuracy, test loss, a multi-line chart of validation accuracy/loss vs. training accuracy/loss, and a confusion matrix. However, due to slight variations in model development, the fine-tuning models omit visualizations of training and validation accuracy and loss.

The dataset for training and testing is available through Kaggle, titled "KAChallenges Series 1: Classifying Math Problems", and consists of 10k training samples and 3k testing samples of math problems labeled with a topic in mathematics (Algebra, Geometry, etc). A preliminary exploratory data analysis (EDA) is conducted to understand the dataset and examine for class balance. Five EDA experiments for vectorization and six for embeddings follow, where hyperparameters for downstream model development are evaluated. These experiments analyze vocabulary composition and the accuracy of a simple model as vocabulary size, sequence length, token length, and other factors are adjusted. All vectorization EDA experiments use a continuous bag-of-words model to see downstream testing accuracy. A continuous bag-of-words (CBOW) model aggregates context representations from the input vector to output a prediction (Geeksforgeeks 2025a). Embedding experiments use a simple logistic regression model to test accuracy when hyperparameters are adjusted.

The first EDA experiment evaluated the vectorized training set of math problems without a custom vocabulary. For comparison, an EDA experiment is conducted in which stopwords are removed. The custom list of stopwords is downloaded from the nltk library and removes

common phrases unlikely to help differentiate the content of a news article, such as “the”, “again”, “could”, and “few”. The architecture of EDA Experiment 2 is carried over to the next experiment, here with the sequence length adjusted iteratively. The fourth EDA Experiment varied vocabulary sizes to evaluate their impact on corpus composition and test accuracy, while holding all other vectorization and modeling parameters constant relative to EDA Experiment 3. The fifth EDA experiment compared CBOW accuracy with and without random oversampling to address class imbalance. Random oversampling is a technique that randomly duplicates underrepresented classes (Brownlee 2021).

The following six experiments sought to adjust parameters relevant to embedding text from the dataset using BERT and MathBERT. EDA Experiments for embeddings mapped the distribution of token lengths when text was embedded with BERT and MathBERT, respectively, to determine the max token length hyperparameter. The following EDA experiments on embeddings identified the best pooling strategy for BERT and MathBERT, which is useful when embeddings are frozen and fed into a Dense layer. The last two experiments for BERT and MathBERT embeddings probed layer depth, which selects the layer used to output embeddings.

After hyperparameter selection through EDA is finalized, experimentation over model development commences. The first five experiments iteratively evaluate the LSTM model's accuracy as the architecture becomes increasingly complex. An LSTM model is an advanced recurrent neural network (RNN) that processes sequential input, and its memory-gating mechanisms enable it to outperform standard RNNs on long sequences (Anantula 2023). In addition to sequential input and short-term hidden-state information, each LSTM cell includes a cell state, a forget gate, an input gate, and an output gate. The cell state carries long-term

memory across the sequence and is updated using the various gates. The forget gate decides which information from the cell state to disregard for the current input. The input gate decides which information to add to the long-term memory cell state, and the output gate decides how much of the cell state to include in the short-term hidden state. Because the cell state is updated through largely additive operations regulated by these gates, this architecture addresses the vanishing/exploding gradient problem in RNNs, where backpropagated gradients can vanish to 0 or explode to very high values.

Experiment 1 is the baseline model, a simple LSTM with unit size, embedding dimension, and output dimension set to 32. The unit size determines the hidden layer size. The larger the unit size, the greater the model's capacity to capture patterns in the hidden state memory, at the potential cost of performance and model fit (Anantula 2023). Experiment 2 introduces early stopping into the architecture to evaluate the impact on model accuracy and performance. Early stopping is a regularization technique that halts training when performance on a selected metric plateaus or worsens, subject to a “patience” parameter that specifies how many epochs to wait before stopping (Brownlee 2019). Experiment 3 adds L2 regularization to the neural network architecture to improve the model's generalizability. L2 regularization is another technique used to reduce overfitting by penalizing features with large weights. Features with lower weights will have an even smaller impact, and features with higher weights will be reduced to improve generalization (Nagpal 2024). Experiment 4 introduces dropout into the architecture, again to decrease model overfitting. Dropout reduces overfitting by randomly dropping layer outputs, with a dropout rate specified during model build (Brownlee 2017). The fifth experiment combined bidirectionality, a technique in which sequential input is processed

both forward and backward, to assess its impact on model accuracy and performance (Geeksforgeeks 2025b).

Experiment 6 introduces a change in methodology, switching from vectorization to embeddings to represent math question text. Here, BERT is used to embed the training dataset, and embeddings are then passed to a simple neural network. BERT, which stands for bidirectional encoder representations from Transformer, uses bidirectional training, an encoder-only Transformer architecture, and masking to understand natural language (HuggingFace n.d.). Largely, embeddings differ from vectorization in that they map token representations into a vector space to model relationships among tokens and thereby assign meaning to them (Metzger 2022). The neural network in Experiment 6 consists of a single hidden layer with 256 nodes and uses a softmax activation for the output. The pooling strategy hyperparameter identified in the EDA is used here to compress the sequential data into a 1D feature array, as required by the Dense layer's input. Experiment 7 continues experimenting with BERT embeddings, this time passing them to the LSTM architecture from experiment 5. Pooling is no longer required in Experiment 7, since the LSTM can process sequential inputs.

Experiment 8 feeds BERT embeddings into a Transformer architecture to evaluate accuracy, model fit, and performance. The Transformer is a neural network architecture introduced by Vaswani et al. in 2018 that uses positional encoding, multi-head self-attention, layer normalization, and a feed-forward neural network to process sequential data (Ferrer 2024). The Transformer in Experiment 8 is an encoder-only architecture that uses two Transformer blocks, four attention heads, an embedding dimension of 64, and a dropout rate of 0.2. The Transformer block count of 2 means that two Transformer blocks, each consisting of attention,

dropout, and layer normalization, are stacked. The multiple self-attention heads enable input tokens to relate to other tokens within a sequence, with more meaningful relations corresponding to closer attention. Positional encoding enables the meaning of order in sequential data to be retained, and layer normalization is a regularization technique that stabilizes training. Since BERT embeddings are produced using an encoder-only architecture, the Transformer omits positional embeddings, as BERT already provides that information. Experiments 9, 10, and 11 repeat the previous three experiments, except that they use MathBERT instead of BERT. MathBERT, developed in 2021 by Peng et al., is a modified BERT architecture trained on scientific journal articles from arXiv and evaluated on formula-topic datasets.

The final two experiments fine-tune BERT and MathBERT, respectively, on the Kaggle dataset. Fine-tuning here involves tokenizing the data with BERT/MathBERT, passing the tokens to a BERT model with a classification head, and setting training hyperparameters. A weight decay of 0.01, a learning rate of 0.00005, and an epoch count of 5 are set. Otherwise, all parameters of the BERT model structure, including layer count, are maintained.

5.0 Results

The key results from all 11 EDA and 13 model experiments are shown in Appendix A, Figures A1 and A2. From a brief analysis of the text sequence composition in Figure A3, it can be observed that the text is in English, with escaped notation using double backslashes and math formulas delineated with dollar signs. Questions vary in length, with some simply asking to solve an equation and others providing more context. This variance in question content must be adequately captured by the “best” model for successful text classification. In a small test of class

balance, the topics “Algebra” and “Geometry” are overrepresented, while “Linear Algebra” and “Abstract Algebra and Topology” are underrepresented.

The first EDA experiment, which vectorized data without custom stopwords, identified 429056 words. Each article contained between 1 and 1022 tokens. The vocabulary consisted of 28324 unique words, the most common of which are shown in Figure A4. The tokens-per-document distribution is right-skewed, with a median around 30 (Figure A5). When used to train a CBOW model, the training time was 5 seconds, and the test accuracy was 0.77.

In EDA Experiment 2, stopwords are removed. With this vectorization architecture, 259411 words with a unique vocabulary of 28188 were found in the corpus. The first 50 vocabulary words differ from EDA Experiment #1 in that many words are now not present. These include “the”, “a”, “to”, and more (Figure A6). However, many tokens are consistent in the vocabulary despite the removal of stop words, such as “number”, “triangle”, and “points”. The distribution of tokens per document is even more right-skewed here, with the median shifted closer to 20 (Figure A7). After testing the vectorization with a CBOW model, the test accuracy was 0.78. Given the slightly higher accuracy after removing stopwords, compared to EDA experiment 1, further experiments will remove stopwords.

In the following experiment, sequence lengths of 50, 100, 200, and 500 are tested. Stopwords are removed for these models. Model accuracy is evaluated at each iteration, and the highest accuracy is 0.78 at a sequence length of 500 (Figure A8). Further experiments will use a sequence length of 500. In EDA experiment 4, vocab sizes of 100, 1000, 10000, and 30000 are iteratively experimented over. The CBOW models achieved the highest accuracy at a vocabulary

size of 30000, with an accuracy of 0.78 (Figure A9). A vocabulary size of 30,000 will be used for vectorization going forward.

The final EDA experiment for vectorization sought to use random oversampling to address class balance and compare model accuracy. Random oversampling achieved an equal distribution of samples across classes, with 2618 samples per class (Figure A10). Here, test accuracy slightly decreased compared to previous experiments, to 0.77. While random oversampling has no discernible effect on CBOW performance, it will be used going forward to simplify comparisons in classification accuracy across classes. Altogether, vectorization EDA resulted in the following hyperparameters being selected for model development: a sequence length of 500, a vocab size of 30,000, with stopword removal and random oversampling for class balance correction.

The following EDA experiments test hyperparameter decisions for BERT and MathBERT embeddings. Experiment 6 embeds the dataset using BERT and shows the distribution of token length (Figure A11). The distribution is right-skewed, with a maximum token length of 3667 but a 99th percentile of 371. Future experiments using BERT embeddings will cap the sequence length at the 95th percentile to capture the majority of sequence structures while omitting outliers. EDA Experiment 7 compares pooling strategies for use when BERT embeddings are passed to a model that doesn't process sequential data, like a simple neural network. Mean and CLS pooling techniques are experimented with, and mean pooling shows the best accuracy at 0.74. Finally, Experiment 7 copies the final architecture of Experiment 6, using mean pooling, except with a layer depth of -2. This experiment found that the highest accuracy occurs at a layer depth of -1.

The final three EDA experiments on embeddings followed the approach used in experiments 6-8, except that they used MathBERT. The distribution of token lengths is exactly the same in Experiment 9 as when evaluated using BERT, with a maximum token length of 3667 but a 99th percentile of 371 (Figure A12). Likewise, Experiment 10 found that mean pooling yielded the highest logistic regression accuracy, at 0.69. Unlike in BERT experiments, Experiment 11 demonstrated that a layer depth of -2 achieved the highest accuracy with MathBERT embeddings.

Relevant hyperparameter decisions are carried over to Experiment 1, a simple LSTM model using vectorization with 32 units. Training time was 26 seconds, and testing accuracy was 0.70. When plotting model validation accuracy/loss against training accuracy/loss, the gap between lines converges and then diverges between epochs 1 and 1.5, with validation accuracy lower than training accuracy at the final epoch (Figure A13). Label 1, “Geometry and Trigonometry,” yielded the highest accuracy at 88.2%, while label “linear algebra” had the lowest at 16.7%. “Linear Algebra” true labels were frequently misclassified as “Algebra” using this model (Figure A14).

Experiment 2 introduces early stopping. Early stopping resulted in a training time of 35.5 seconds. Testing accuracy increases slightly to 0.72, compared to the previous model. Model fit still suffers from overfitting, with training and validation accuracies diverging and training accuracy exceeding validation accuracy (Figure A15). As in Experiment 1, the correlation matrix shows the highest accuracy for the label “Geometry and Trigonometry” at 88.3% and the lowest for “Linear Algebra” at 16.7%, with “Linear Algebra” frequently misclassified as “Algebra” (Figure A16).

Experiment 3 aims to evaluate the effect of L2 regularization on the LSTM model architecture. The training time was 132.8 seconds, and testing accuracy increased slightly to 0.73. Figure A17 shows that the model fit has improved due to regularization. Here, validation and training accuracies are closer, with a difference of 8.07% at the final epoch. The confusion matrix more closely resembles that from Experiment 2, with the label “Linear Algebra” at the lowest classification of 30% (Figure A18). L2 regularization appears to have successfully reduced model fit issues, while maintaining model performance and accuracy from previous iterations.

Experiment 4 continues to address model overfitting by introducing dropout. Training time and epoch count decreased for this model to 83 seconds. Test accuracy decreased to 0.69 with this model, and the gap between training and validation accuracy also decreased to 2.87% (Figure A19). There are no notable differences in the confusion matrix for this model compared to previous ones (Figure A20). Again, overfitting was successfully addressed here, although at the expense of model accuracy.

The final LSTM model experiments with bidirectionality. Testing accuracy improves very slightly to 0.7. Training accuracy and validation accuracy have widened at the last epoch compared to the previous experiment, now at 4.51% (Figure A21). The confusion matrix shows little difference in label-wise accuracy compared to previous experiments (Figure A22). Altogether, while the added model complexity introduced by bidirectionality may have slightly improved test accuracy, the trade-off was worse model generalizability.

Experiment 6 entails a switch in architecture to using BERT embeddings and feeding them into a Dense layer. Embeddings are loaded and configured using hyperparameters selected

in EDA. Time to train the simple neural network, after embeddings are generated, is 11 seconds. Here, we see comparable test accuracy to previous experiments, at 0.74. However, model generalizability has improved over previous experiments, with the gap between validation and training accuracy at the last epoch reduced to 1.08% (Figure A23). The confusion matrix shows improved classification accuracy across the board, with “Geometry and Trigonometry” now being classified at 92.5% and “Linear Algebra” at 40% (Figure A24). The use of context-aware embeddings has so far been shown to slightly increase testing accuracy while improving model generalizability.

The following experiment feeds BERT embeddings into the same LSTM architecture from Experiment 5. Here, the training time is increased 123 seconds. This model shows the highest accuracy of models developed thus far, at 0.78. However, the line chart of training and validation accuracy indicates overfitting, with a final-epoch percent difference of 6.7% (Figure A25). Surprisingly, the confusion matrix shows that “Linear Algebra” here is entirely unclassified, with 0% correct classifications (Figure A26). Otherwise, the confusion matrix is similar to those from previous experiments. Using BERT embeddings instead of vectorization increases LSTM model accuracy but also introduces trade-offs in model generalizability and classification performance for underrepresented classes.

Experiment 8 feeds BERT embeddings into an encoder-only Transformer. Training time for this model is the highest seen thus far, at 205 seconds, despite using only 5 epochs. Unfortunately, the increase in training time did not translate into higher accuracy, with a test accuracy of only 0.7. Model fit is improved using BERT + Transformer compared to all other models, with a last-epoch difference in validation and training accuracy of 0.45% (Figure A27).

Again, the confusion matrix shows surprising results: both underrepresented categories, “Linear Algebra” and “Abstract Algebra and Topology,” are completely misclassified, with 0% of correct classifications (Figure A28). BERT + Transformer may result in better model generalizability compared to previous iterations; however, classification performance suffers, both over all classes and especially for underrepresented ones.

Experiment 9 repeats experiment 6, except this time using MathBERT embeddings with a layer index of -2. Training time is 7 seconds, and testing accuracy is 0.75. This is a higher accuracy than in Experiment 6. The gap in validation and training accuracy is wide, at 8.7% at the last epoch (Figure A29). The confusion matrix here is similar to that of Experiment 6, with “Geometry and Trigonometry” at 89.2% and “Linear Algebra” at 30% (Figure A30). Compared to a simple neural network using BERT embeddings, MathBERT achieves higher accuracy but poorer generalization.

The following experiment repeats Experiment 7, again here using MathBERT and feeding into an LSTM with bidirectionality, 32 Units, L2 Regularization, and Dropout. Time to train is 97 seconds, slightly faster than the LSTM + BERT architecture. Test accuracy is 0.75, slightly worse than the LSTM + BERT model. Model generalizability is worse than that of the LSTM + BERT counterpart, at 15.9% (Figure A31). As seen with BERT and MathBERT embeddings, the underrepresented class “Linear Algebra” is entirely misclassified (Figure A32).

Experiment 11 is a copy of Experiment 8, here using MathBERT embeddings instead of BERT embeddings to feed into a Transformer architecture. As with BERT + Transformer, training MathBERT + Transformer takes longer than other model architectures, at 321 seconds. Testing accuracy is higher here compared to BERT + Transformer, at 0.76, and the gap between

training and validation accuracy is only 3.1% at the last epoch (Figure A33). However, the class "Linear Algebra" is still wholly misclassified with this model (Figure A34). Altogether, this model shows an improvement over the BERT + Transformer architecture in terms of accuracy, with only a slightly wider gap between training and validation accuracy.

The final two models fine-tune BERT and MathBERT, respectively, using the Kaggle dataset. Fine-tuning the BERT dataset took 718 seconds, the longest training time to date. However, testing accuracy is also at its highest yet, at 0.83. The difference between training and validation accuracy at the latest epoch is 12.5%, relatively high compared to other models seen thus far. The confusion matrix shows results similar to the best LSTM models, with "Geometry and Trigonometry" at 93.7% and "Linear Algebra" at 33.3% (Figure A35). Unlike passing BERT embeddings into an LSTM or Transformer model, the fine-tuned BERT successfully classifies at least some samples for underrepresented classes like "Linear Algebra" and "Abstract Algebra and Topology". The fine-tuned MathBERT is very similar to the fine-tuned BERT, with a shorter training time of 416 seconds, a lower test accuracy of 0.809, and a final-epoch gap of 12.7% between validation and test accuracy. Classification matrices are similar between fine-tuned BERT and MathBERT; classification accuracy for the underrepresented class "Abstract Algebra and Topology" is substantially higher with MathBERT, at 84.6% (Figure A36).

6.0 Conclusions

The 13 experiments conducted in this study evaluated the trade-offs among model accuracy, training time, and model fit across varying sequential neural network topologies, benchmarked on the "KAChallenges Series 1: Classifying Math Problems" dataset for use in an auto-tagger application for education. Broadly speaking, fine-tuning experiments achieved the

best accuracy across the board, with fine-tuned BERT achieving the highest at 82.92%. Using frozen BERT or MathBERT embeddings typically yielded higher accuracy than vectorized data, with an LSTM model using BERT embeddings achieving the highest accuracy among non-fine-tuned models. However, most models, including those using frozen embeddings, performed poorly on underrepresented classes such as “Linear Algebra” while improving accuracy on better-represented categories such as “Geometry and Trigonometry”. No model over the entire study was able to classify “Linear Algebra” above 40%. Overfitting was prevalent in most experiments, including those that achieved the highest accuracy. As expected, training time was the longest for the fine-tuned models.

Surprisingly, experiments using MathBERT embeddings or fine-tuning MathBERT yielded performance comparable to simply using BERT. The biggest improvement was in the classification of underrepresented labels, where the fine-tuned MathBERT outperformed the fine-tuned BERT in classifying "Abstract Algebra and Topography". However, the overall test accuracy of fine-tuned MathBERT was lower. Since MathBERT is largely pre-trained on scientific journal articles, the advantage of using MathBERT for math problem classification may have been minimal.

The management problem of auto-tagging for education requires an auto-tagger to be as accurate as possible and to generalize well over large datasets that may suffer from data scarcity due to rare labels. Because of these requirements, the fine-tuned BERT model is selected as the “best” from this experiment. This model had the highest accuracy, with an almost 10-point increase over most other models. The fine-tuned BERT classified the lower-represented classes better compared to other models, with only the fine-tuned MathBERT doing better at classifying

“Abstract Algebra and Topology”. While training fine-tuned models is time-consuming and may hinder quick updates to production, accuracy remains the most important metric for auto-tagger use.

However, many improvements to the model are needed before it can reach production readiness, particularly to improve accuracy and performance on underrepresented classes. Class balance may be further addressed using differing augmentation techniques, such as synonym replacement, random swap, or even LLM-generated sampling. Ideally, addressing class imbalance will improve the classification of rare labels. Further enhancements to hyperparameter tuning and technique selection can be attempted, including larger unit sizes, transfer learning, and more. Different architectures and machine learning models may be tested, such as a 1D-CNN or BERT Embeddings + XGBoost classifier. Other embedding methods can be tried, such as RoBERTa, ModernBERT, or SentenceTransformer. Additionally, an ensemble of neural networks may provide further gains in prediction accuracy. These findings emphasize the need to assess model complexity when designing deep learning models, especially when targeting accuracy and model fit for an educational auto-tagger.

References

Ali Habibullah, KAChallenges, and Mohamed Eltayeb. KAChallenges Series 1: Classifying Math Problems. https://kaggle.com/competitions/classification-of-math-problems-by-kasut-academy, 2025. Kaggle.

Anantula, Arya. 2023. "Deep Learning Notes: RNNs and More." HackMD. December 27, 2023. https://hackmd.io/@Arya333/B1-rWZrwT

Brownlee, Jason. 2017. "Dropout for Regularizing Deep Neural Networks." *MachineLearningMastery.com*. https://machinelearningmastery.com/dropout-for-regularizing-deep-neural-networks/

Brownlee, Jason. 2019. "Early Stopping to Avoid Overtraining Neural Network Models." *MachineLearningMastery.com*. https://machinelearningmastery.com/early-stopping-to-avoid-overtraining-neural-network-models/

Brownlee, Jason. 2021. "Random Oversampling and Undersampling for Imbalanced Classification." *MachineLearningMastery.com*, January 5, 2021. https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/

Ferrer, Josep. 2026. "How Transformers Work: A Detailed Exploration of Transformer Architecture." DataCamp, Feb 27, 2026. https://www.datacamp.com/tutorial/how-transformers-work

GeeksforGeeks. 2025a. "Continuous Bag of Words (CBOW) in NLP." GeeksforGeeks. July 23, 2025. https://www.geeksforgeeks.org/nlp/continuous-bag-of-words-cbow-in-nlp/

GeeksforGeeks. 2025b. "Bidirectional Recurrent Neural Network." *GeeksforGeeks*. December 17, 2025. https://www.geeksforgeeks.org/deep-learning/bidirectional-recurrent-neural-network/

Grévisse, Christian, Claude Braun, and José Batista Da Costa. 2025. "Autotag & Tagmap: Llm-powered Moodle Plugins for Pedagogical Alignment Checks". SN Computer Science 6 (7). https://doi.org/10.1007/s42979-025-04348-9.

Hugging Face. n.d. "BERT." Transformers Documentation. Accessed March 15, 2026. https://huggingface.co/docs/transformers/en/model_doc/bert

Lee, Hyun Seung, Seungtaek Choi, Yunsung Lee, Hyeongdon Moon, Shinhyeok Oh, Myeongho Jeong, Hyojun Go, and Christian Wallraven. 2023. "Cross Encoding as Augmentation:

Towards Effective Educational Text Classification". In , 2184–95. https://doi.org/10.18653/v1/2023.findings-acl.137.

Metzger, Sascha. 2022. "A Beginner's Guide to Tokens, Vectors, and Embeddings: How Do You Create Them?" Medium. https://medium.com/@saschametzger/what-are-tokens-vectors-and-embeddings-how-do-you-create-them-e2a3e698e037

Nagpal, Anuja. 2024. "L1 and L2 Regularization Methods, Explained." *Built In*, October 3, 2024. https://builtin.com/data-science/l2-regularization

Peng, Shuai, Ke Yuan, Liangcai Gao, and Zhi Tang. "Mathbert: A pre-trained model for mathematical formula understanding." *arXiv* preprint arXiv:2105.00377 (2021).

Wang, Xiaolu, Ziqi Ding, and Liangyu Chen. "Automatic tagging of knowledge points for k12 math problems." *arXiv* preprint arXiv:2208.09867 (2022).

Appendix A

Experiment	Experiment Description	Testing Accuracy (percent)
1	Vectorization without removing stopwords	0.77
2	Vectorization with stopwords removed	0.78
3a	Vectorization with stopwords removed, sequence length of 50	0.77
3b	Vectorization with stopwords removed, sequence length of 100	0.78
3c	Vectorization with stopwords removed, sequence length of 200	0.78
3d	Vectorization with stopwords removed, sequence length of 500	0.78
4a	Vectorization with stopwords removed, sequence length of 500, vocab size of 100	0.63
4b	Vectorization with stopwords removed, sequence length of 500, vocab size of 1000	0.76
4c	Vectorization with stopwords removed, sequence length of 500, vocab size of 10000	0.78

4d	Vectorization with stopwords removed, sequence length of 500, vocab size of 30000	0.78
5	Vectorization with stopwords removed, sequence length of 500, vocab size of 30000, Random Oversampling	0.77
6	Analysis of BERT token length	-
7a	BERT Embeddings, 211 sequence length, Mean pooling strategy, -1 layer depth	0.74
7b	BERT Embeddings, 211 sequence length, cls pooling strategy, -1 layer depth	0.68
8	BERT Embeddings, 211 sequence length, mean pooling strategy, -2 layer depth	0.71
9	Analysis of MathBERT token length	-
10a	MathBERT Embeddings, 211 sequence length, Mean pooling strategy, -1 layer depth	0.69
10b	MathBERT Embeddings, 211 sequence length, cls pooling strategy, -1 layer depth	0.59
11	MathBERT	0.69

	Embeddings, 211 sequence length, mean pooling strategy, -2 layer depth	

Figure A1. Key Results from 11 EDA Experiments

Experiment	Model Description	Testing Accuracy (percent)	Testing Loss	Training Time (seconds)	Percent Difference, Training Accuracy to Validation Accuracy
1	LSTM, 32 Units	70.33	1.38	26.44	9.70
2	LSTM, 32 Units, Early Stopping	72.24	1.35	35.50	8.61
3	LSTM, 32 Units, Early Stopping, L2 Regularization	72.50	1.40	132.75	8.07
4	LSTM, 32 Units, Early Stopping, L2 Regularization, Dropout	68.76	1.53	82.77	2.87
5	LSTM, 32 Units, Early Stopping, L2 Regularization, Dropout, Bidirectionality	70.20	1.42	86.11	4.51
6	Neural Network, 1 layer, 256 nodes, BERT embeddings	73.98	0.736	10.75	1.08
7	LSTM, 32 Units, Early Stopping, L2 Regularization, Dropout, Bidirectionality, BERT	78.02	0.94	122.96	6.65

	embeddings				
8	Transformer, 2 Transformer Blocks, 4 Attention Heads, Dropout, Encoder-Only, BERT Embeddings	69.88	0.89	204.99	0.45
9	Neural Network, 1 layer, 256 nodes, MathBERT embeddings	75.07	0.71	6.69	8.68
10	LSTM, 32 Units, Early Stopping, L2 Regularization, Dropout, Bidirectionality, MathBERT embeddings	75.30	1.15	97.11	15.9
11	Transformer, 2 Transformer Blocks, 4 Attention Heads, Dropout, Encoder-Only, MathBERT Embeddings	75.82	73.14	320.52	3.08
12	BERT, fine-tuned over Kaggle dataset	82.92	0.56	718.18	12.52
13	MathBERT, fine-tuned over Kaggle dataset	80.85	0.58	416.43	12.69

Figure A2. Key Results from 13 Model Architecture Experiments

```
Sample question for topic Algebra
["b'Solve -80 = v + 4*z - 100, -4*v - 2*z + 122 = 0 for v.\\n'"]

Sample question for topic Geometry and Trigonometry
['An isosceles right triangle $ABC$ ($\\angle B-$ right angle), with an area of $4+2\\sqrt{2}$, is inscribed in a circle. Point $D$ lies on this circle, and the chord $BD$ equa

Sample question for topic Calculus and Analysis
['4. Let $k$ be the number of real roots of the equation $|x|^{\\frac{1}{4}}+|x|^{\\frac{1}{2}}-\\cos x=0$, then $k=$']

Sample question for topic Probability and Statistics
['Two dice appear to be normal dice with their faces numbered from $1$ to $6$ , but each die is weighted so that the probability of rolling the number $k$ is directly proportio

Sample question for topic Number Theory
['445. Find all pairs of natural numbers whose sum is 288, and the greatest common divisor is 36.']

Sample question for topic Combinatorics and Discrete Math
['A math competition consists of three problems, each of which receives an integer score from 0 to 7. For any two competitors, we know that there is at most one problem on whic

Sample question for topic Linear Algebra
["b'Solve -3*m + 25 = -4*o, 3*m - 251762*o = -251756*o + 27 for m.\\n'"]

Sample question for topic Abstract Algebra and Topology
['Prove that any two nonabelian groups of order 21 are isomorphic.']
```

Figure A3. Samples of "KAChallenges Series 1: Classifying Math Problems" dataset, for Visual Analysis

```
['' '[UNK]' 'the' 'of' 'a' 'and' 'is' 'in' 'b' 'that' 'to' 'c' 'are' 'for'
 '1' 'number' 'find' 'be' '2' 'n' 'd' 'x' 'with' 'on' 'numbers' 'if' '3'
 'all' 'at' 'let' 'each' 'then' 'positive' 'two' 'by' 'points' 'such' '4'
 'what' 'from' 'an' 'value' 'triangle' 'one' 'point' '5' 'm' 'as'
 'integer' 'integers']
```

Figure A4. Top 50 Vocab Words for Vectorization, EDA Experiment 1

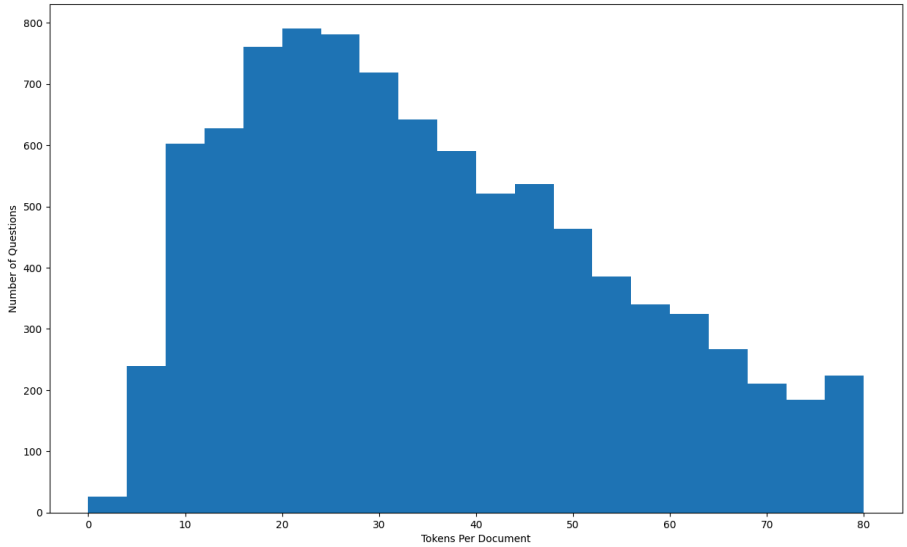

Figure A5. Tokens per Document Distribution, EDA Experiment 1

```
['' '[UNK]' 'b' 'c' '1' 'number' 'find' '2' 'n' 'x' 'numbers' '3' 'let'
 'positive' 'two' 'points' '4' 'value' 'triangle' 'point' 'one' '5'
 'integer' 'integers' 'p' 'qquad' 'e' 'given' 'many' 'sum' '0' 'real' '6'
 'angle' 'three' 'k' 'times' 'area' '10' 'determine' 'square' 'line'
 'first' '8' 'circle' 'equation' '7' 'cdot' 'side' 'sequence']
```

Figure A6.Top 50 Vocab Words for Vectorization, EDA Experiment 2

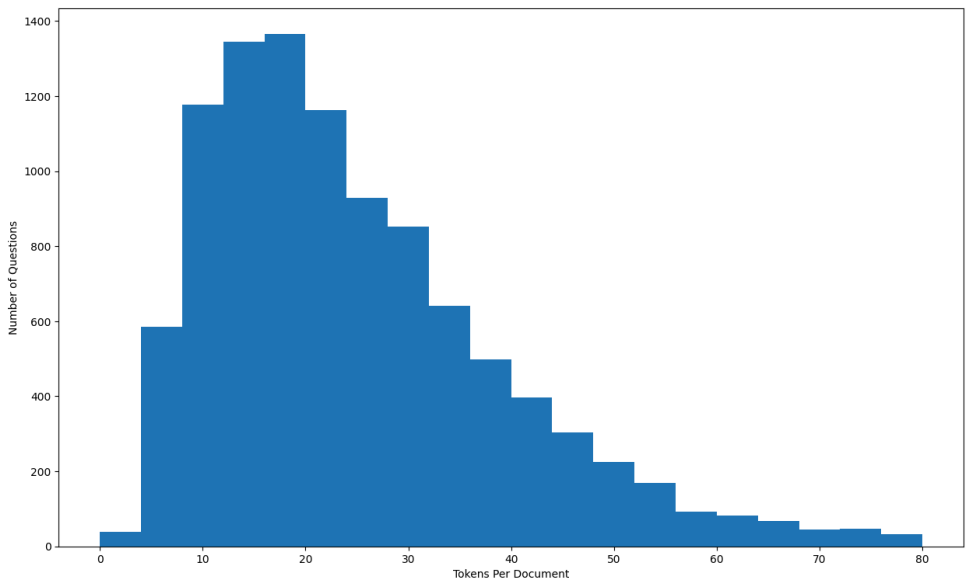

Figure A7. Tokens per Document Distribution, EDA Experiment 2

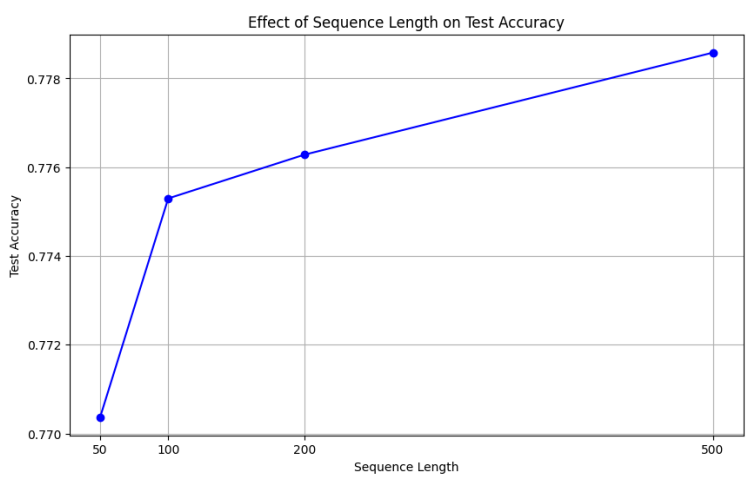

Figure A8. CBOW Accuracy for Varying Sequence Lengths, EDA Experiment 3

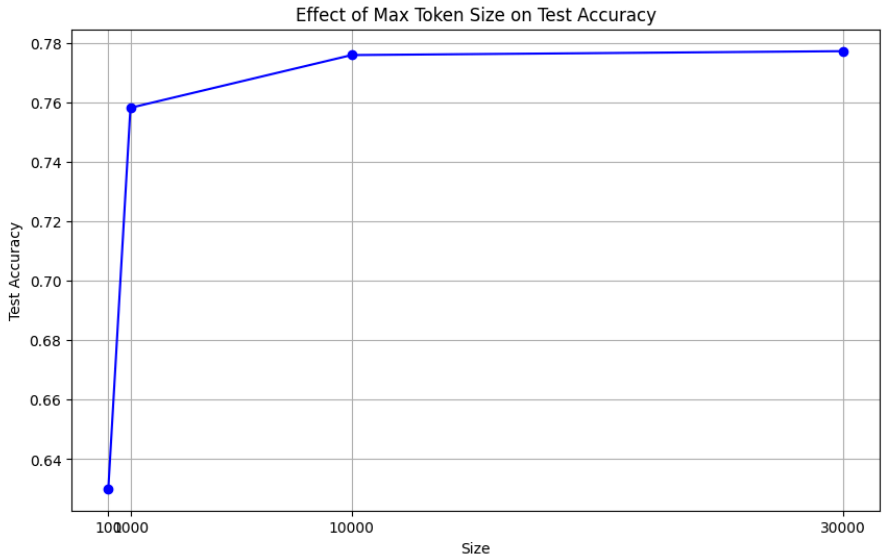

Figure A9. CBOW Accuracy for Varying Vocabular Sizes, EDA Experiment 4

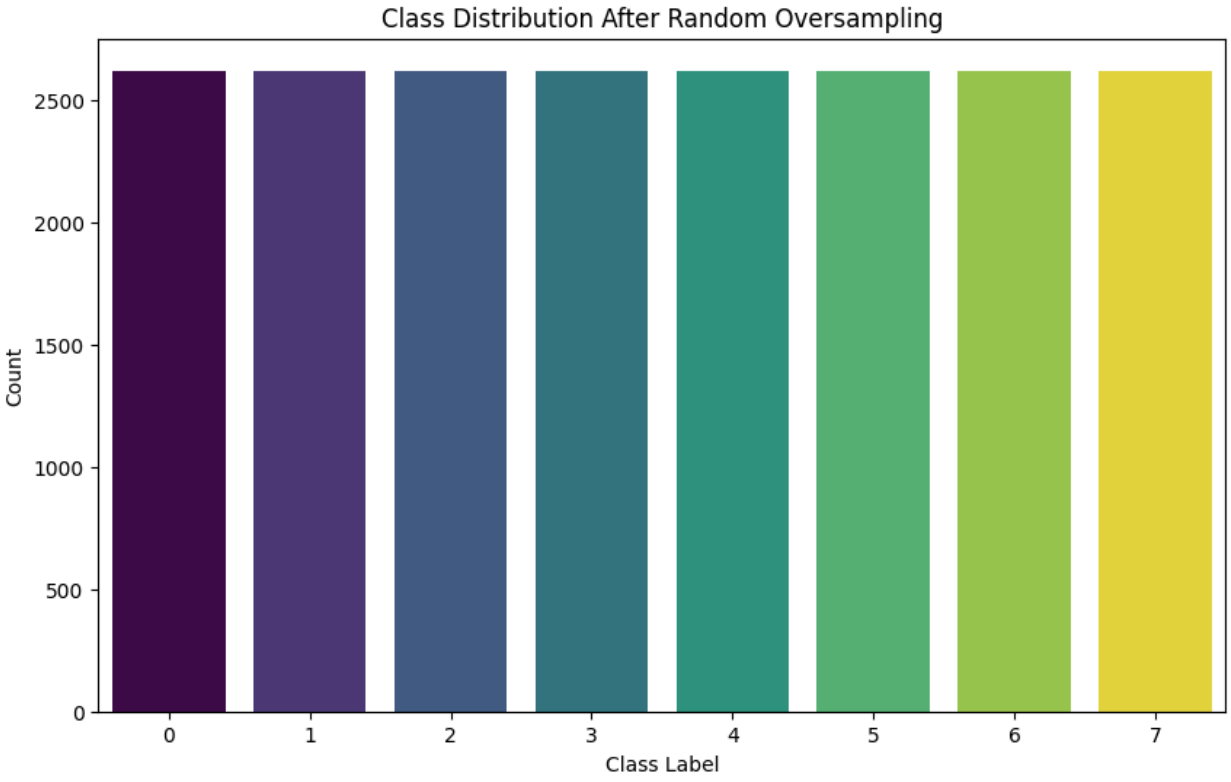

Figure A10. Class Distribution After Class Balance, EDA Experiment 5

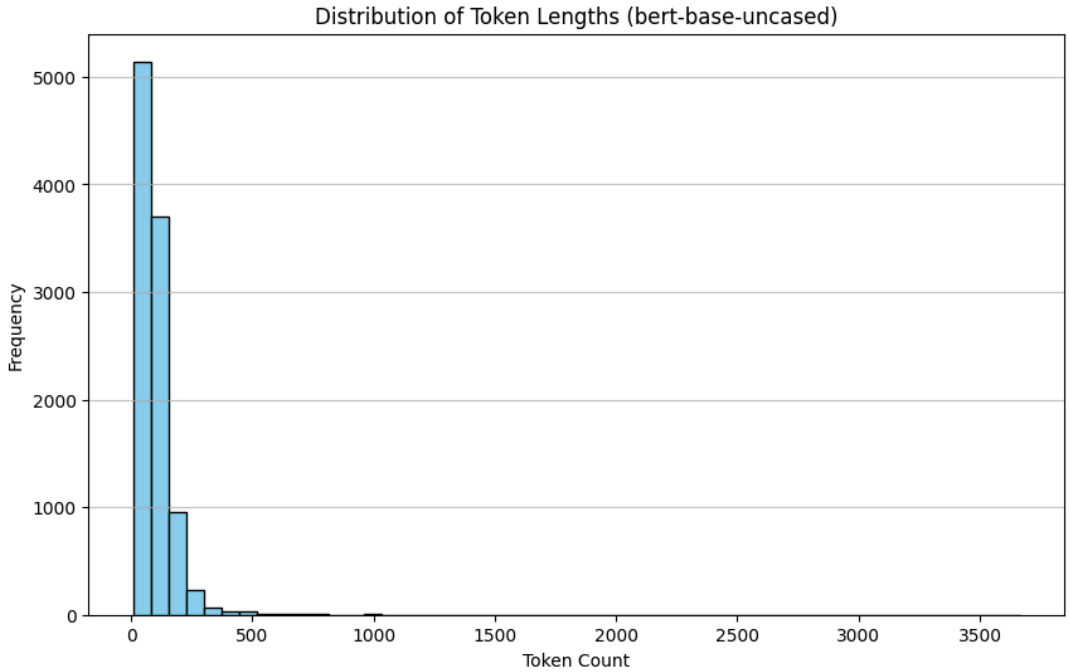

Figure A11. Distribution of Token Lengths after BERT Embedding, EDA Experiment 6

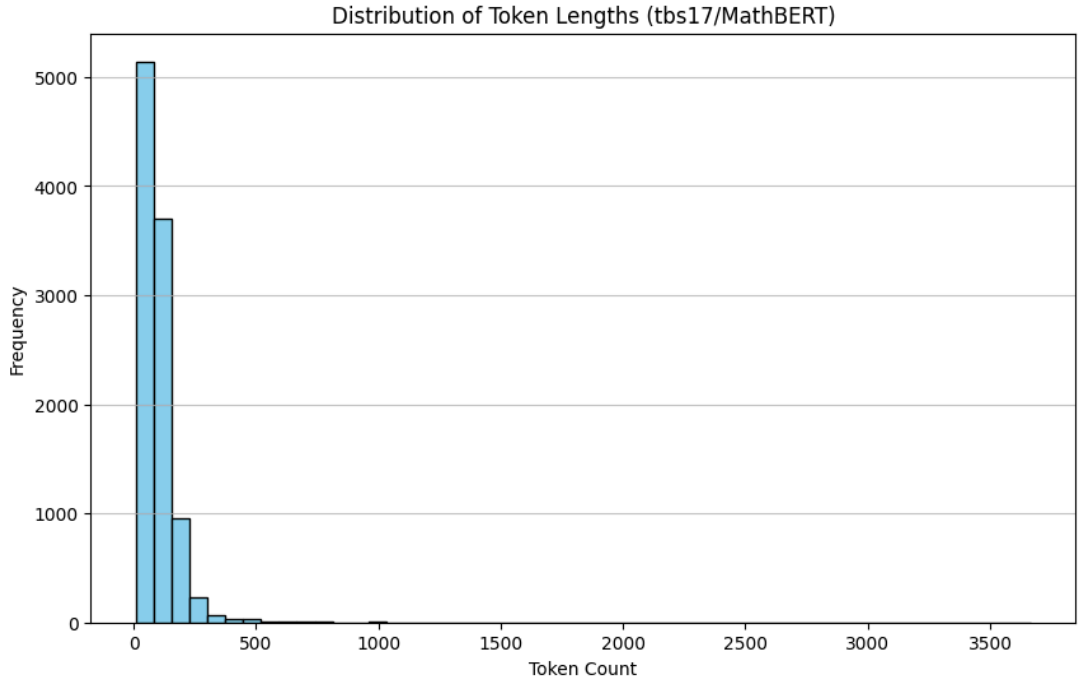

Figure A12. Distribution of Token Lengths after BERT Embedding, EDA Experiment 9

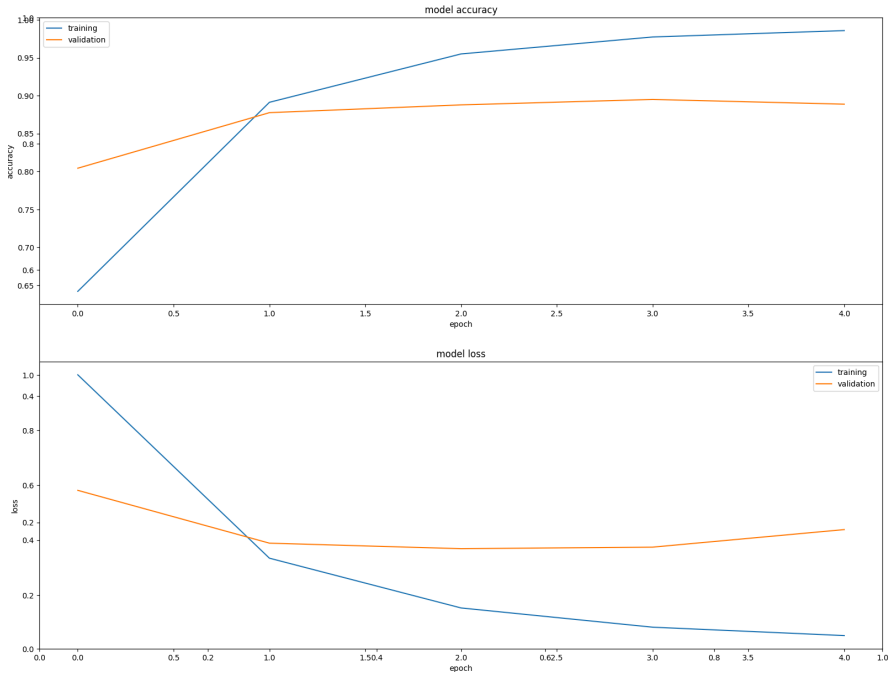

Figure A13. Training and Validation Model Accuracy and Loss, Experiment 1

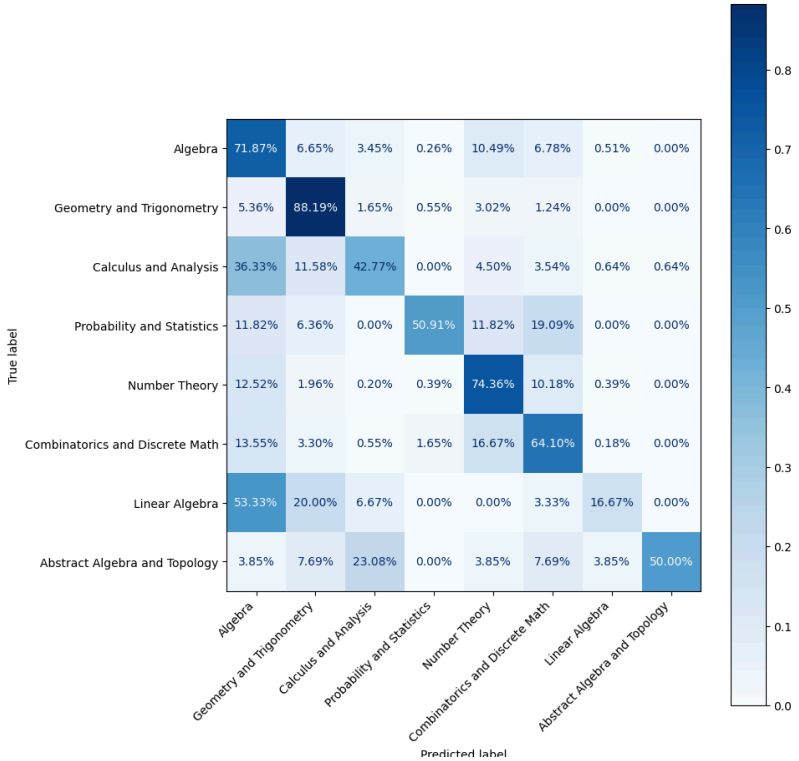

Figure A14. Confusion Matrix, Experiment 1

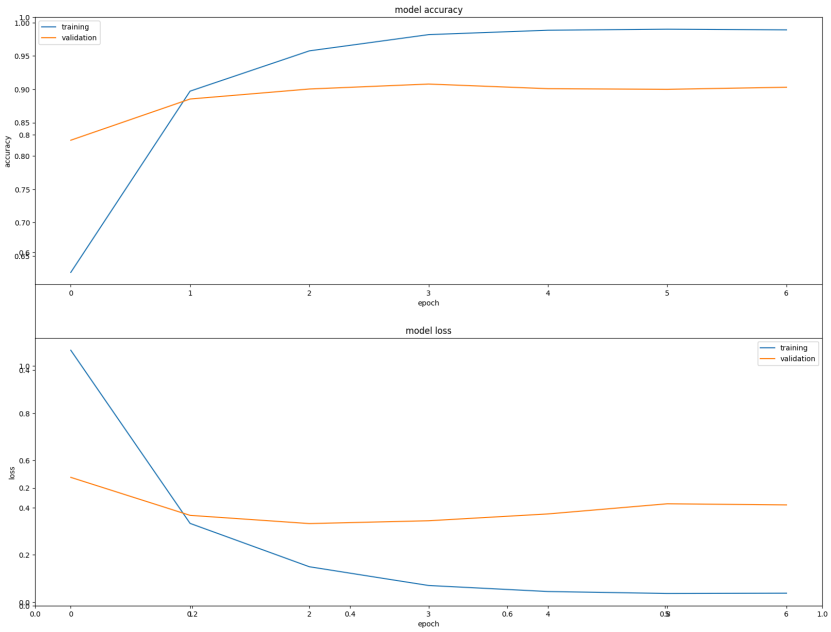

Figure A15. Training and Validation Model Accuracy and Loss, Experiment 2

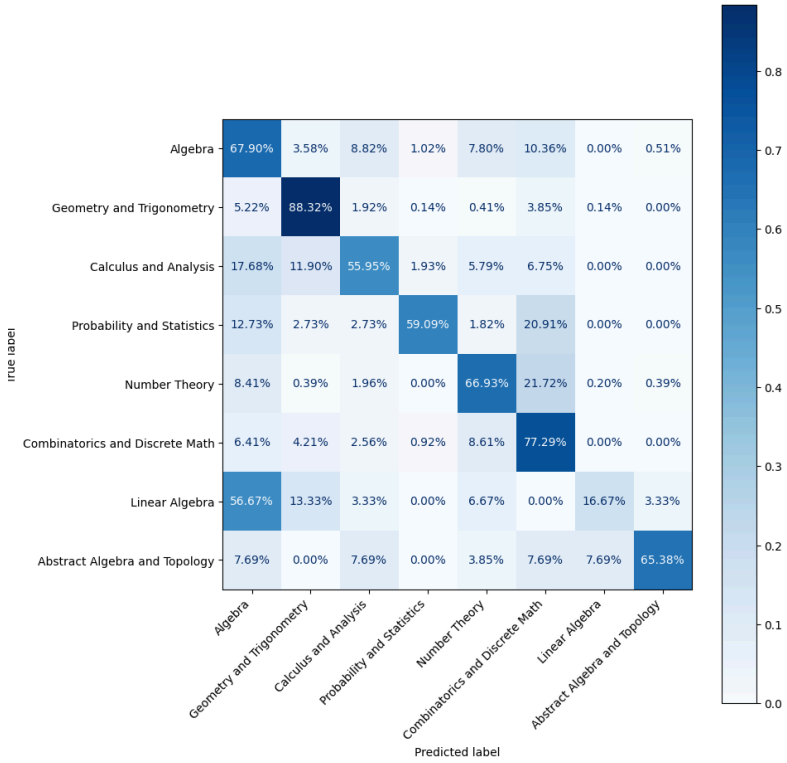

Figure A16. Confusion Matrix, Experiment 2

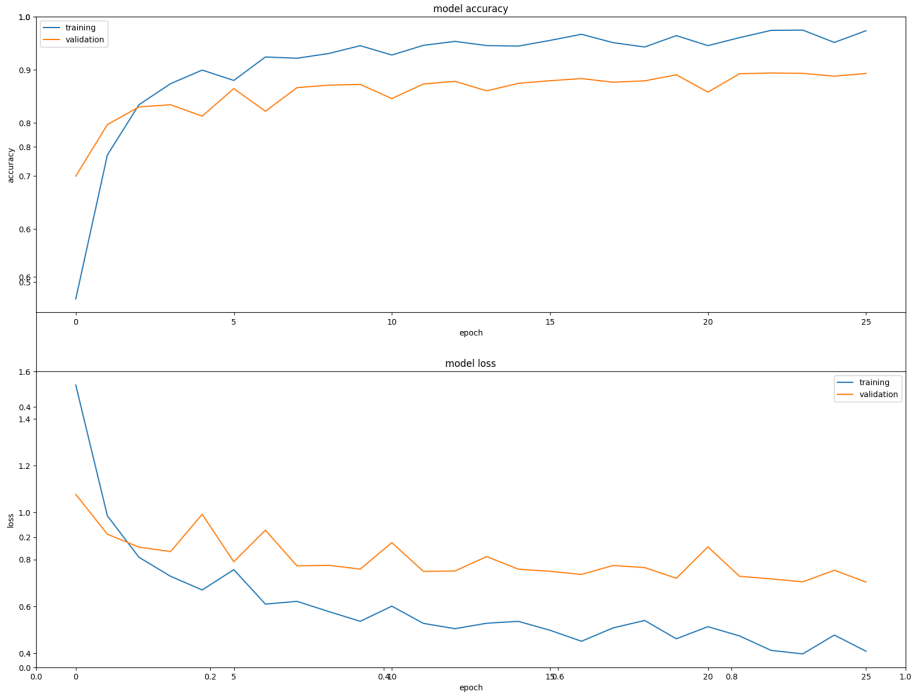

Figure A17. Training and Validation Model Accuracy and Loss, Experiment 3

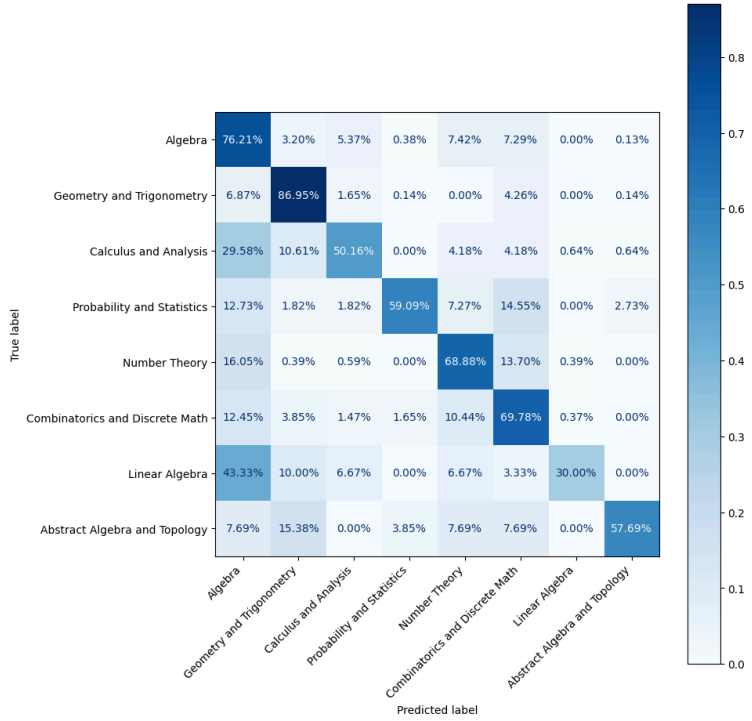

Figure A18. Confusion Matrix, Experiment 3

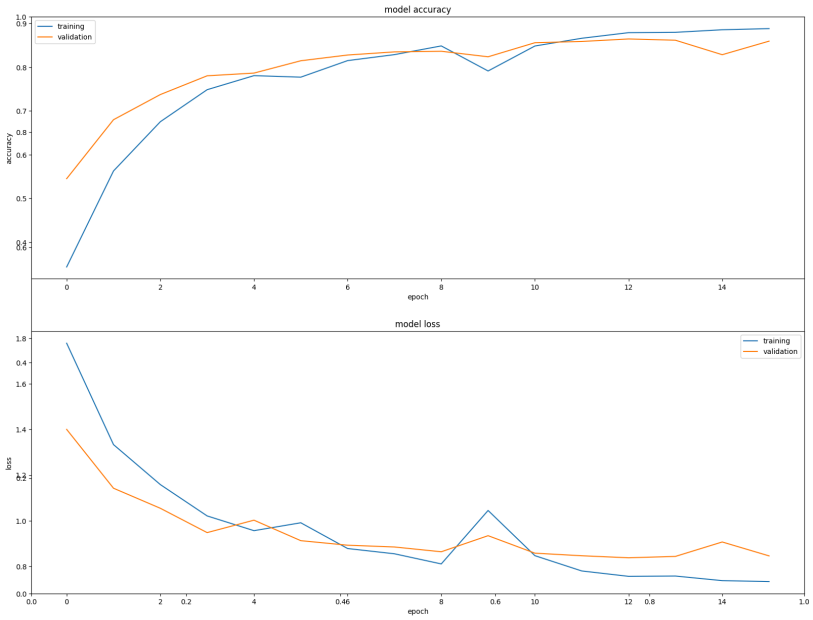

Figure A19. Training and Validation Model Accuracy and Loss, Experiment 4

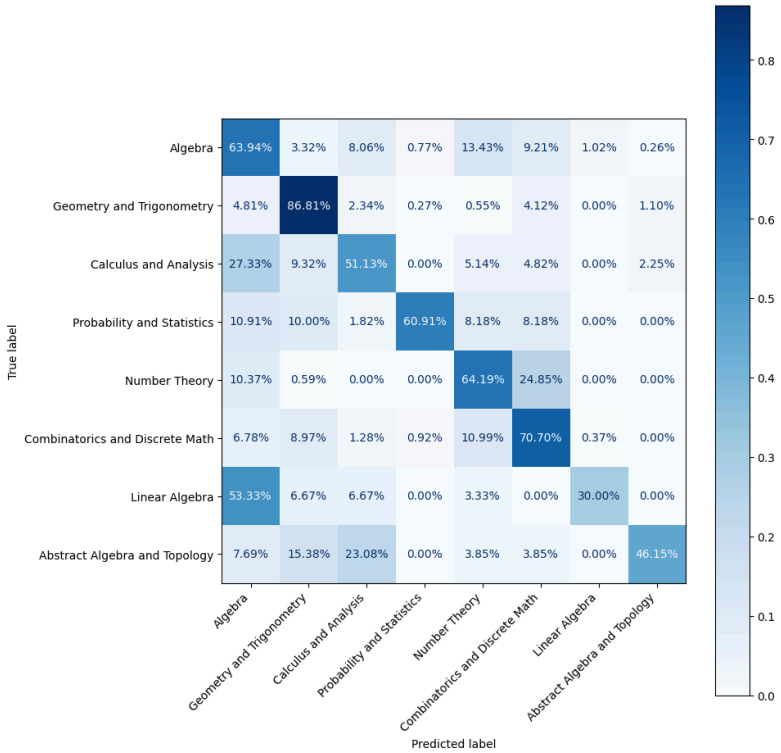

Figure A20. Confusion Matrix, Experiment 4

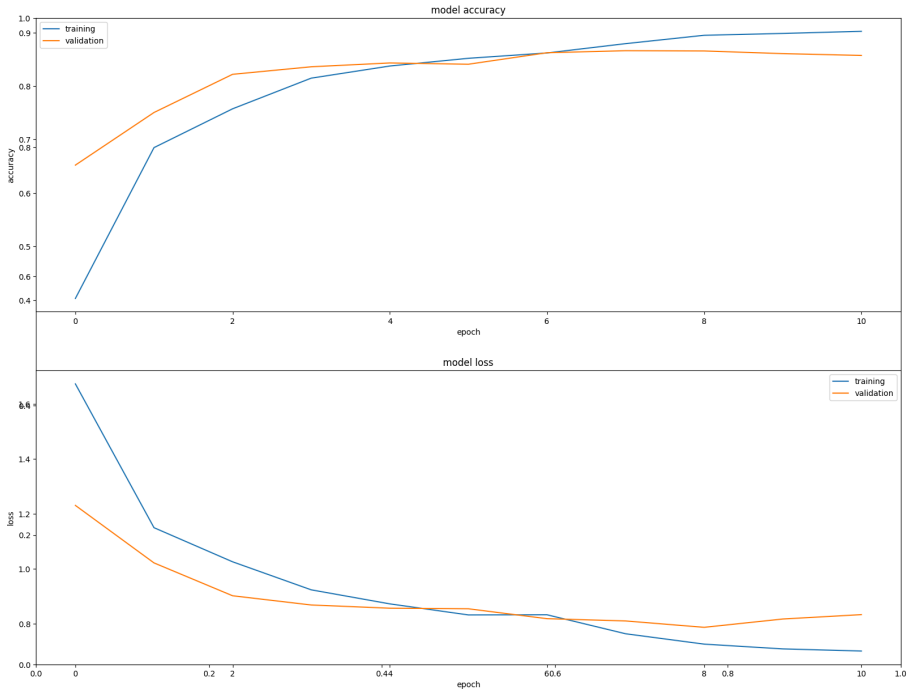

Figure A21. Training and Validation Model Accuracy and Loss, Experiment 5

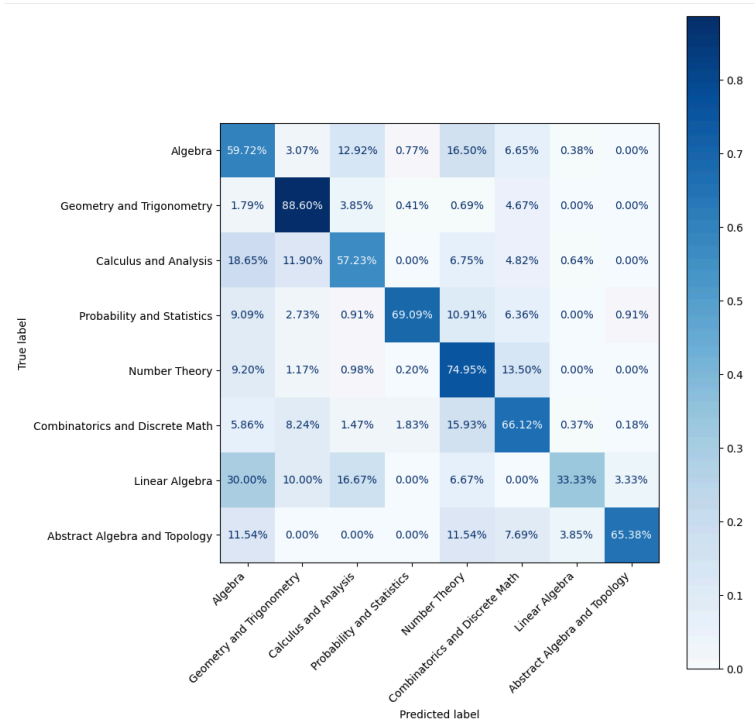

Figure A22. Confusion Matrix, Experiment 5

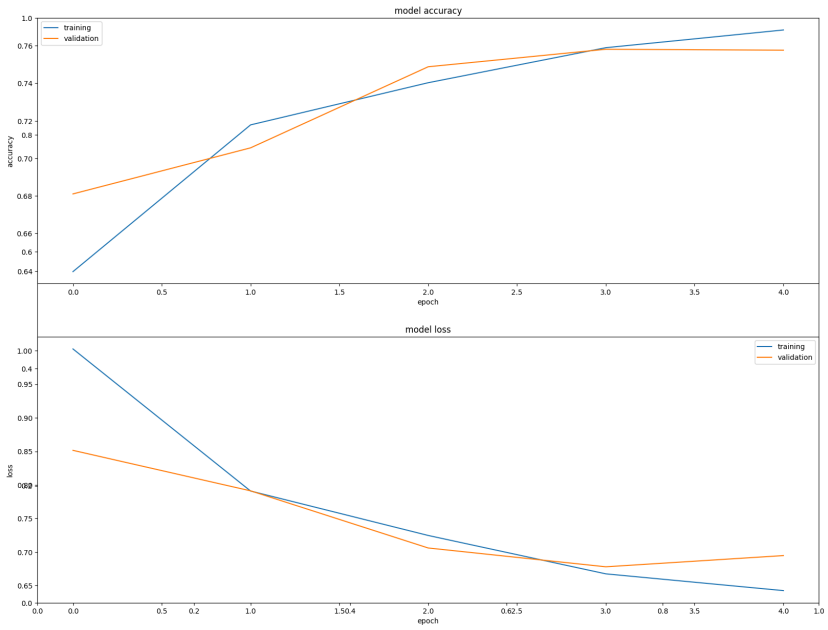

Figure A23. Training and Validation Model Accuracy and Loss, Experiment 6

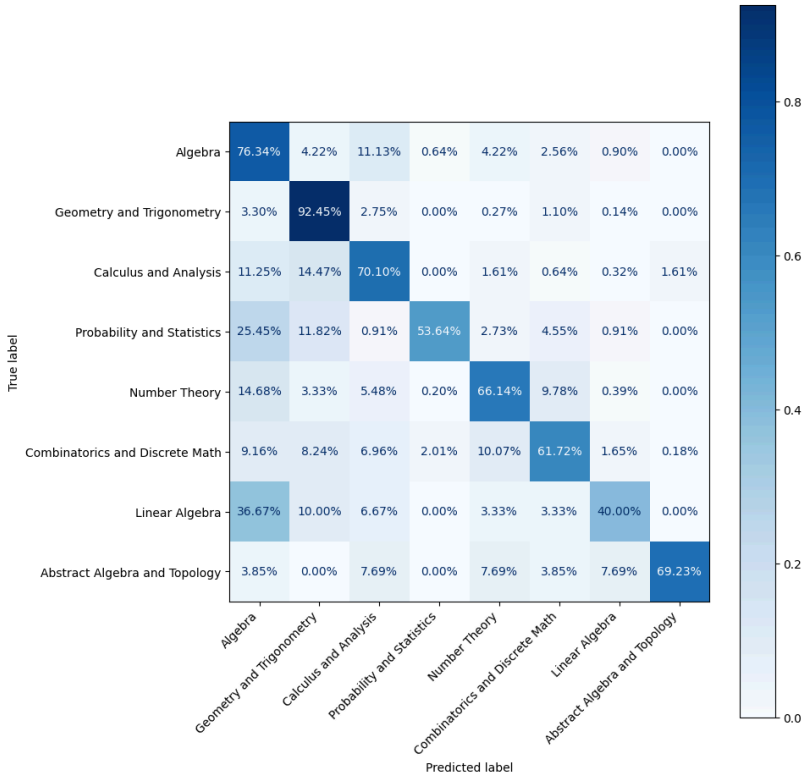

Figure A24. Confusion Matrix, Experiment 6

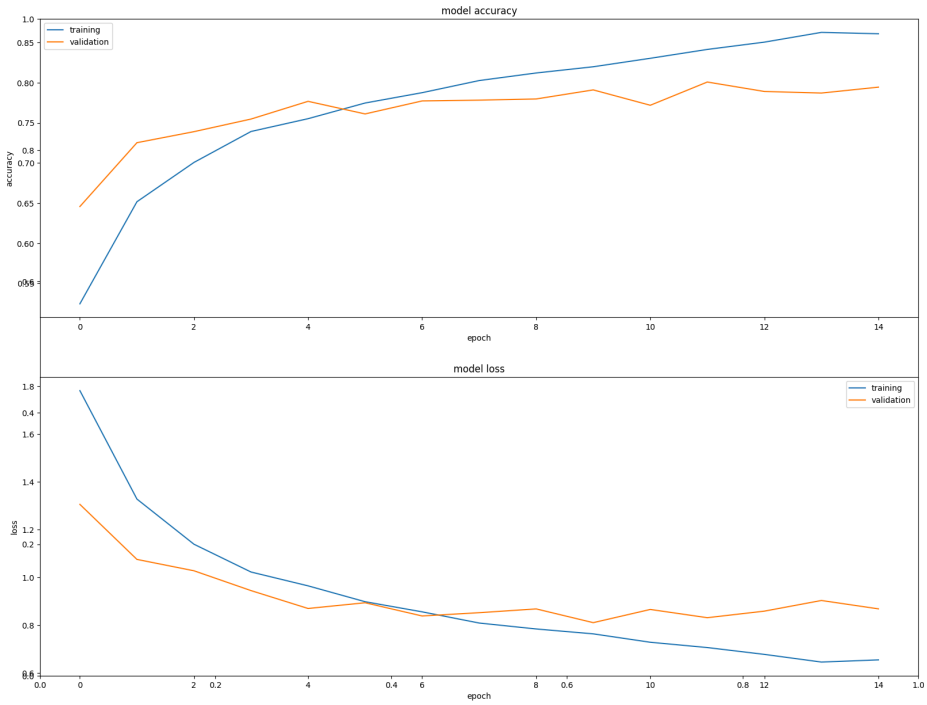

Figure A25. Training and Validation Model Accuracy and Loss, Experiment 7

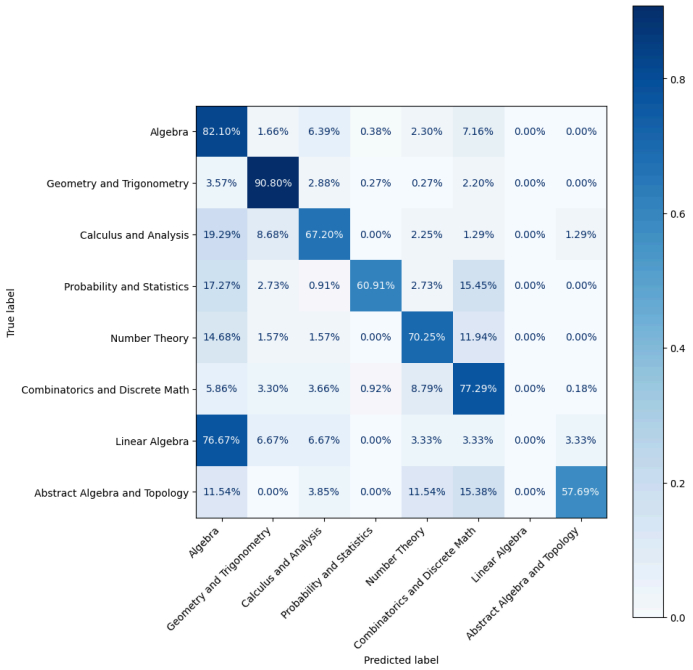

Figure A26. Confusion Matrix, Experiment 7

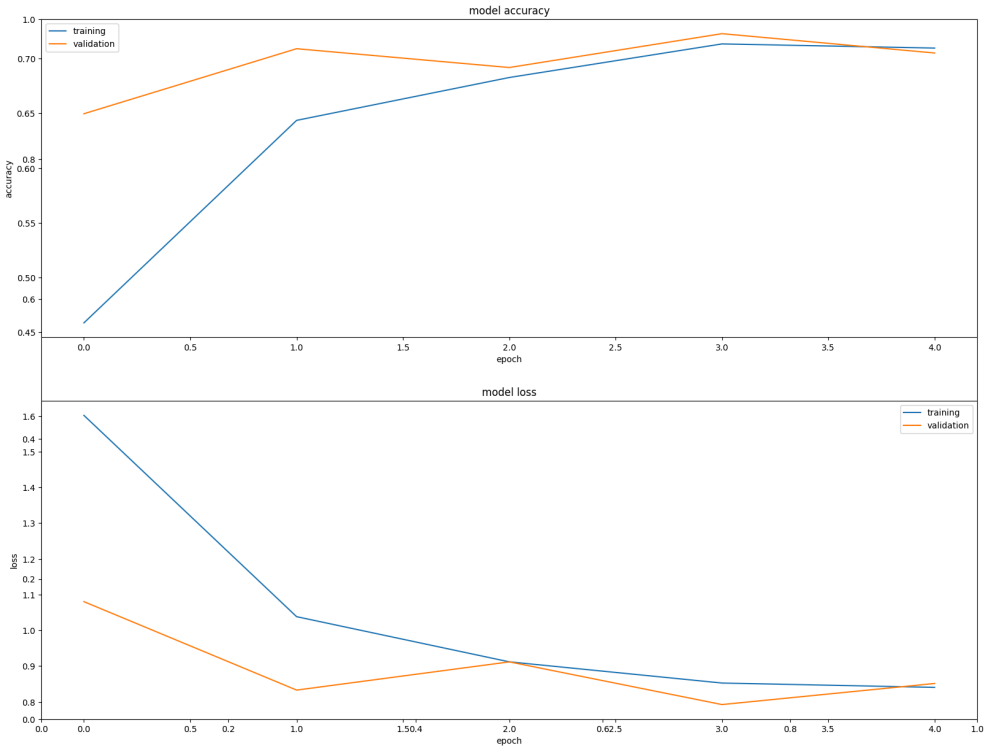

Figure A27. Training and Validation Model Accuracy and Loss, Experiment 8

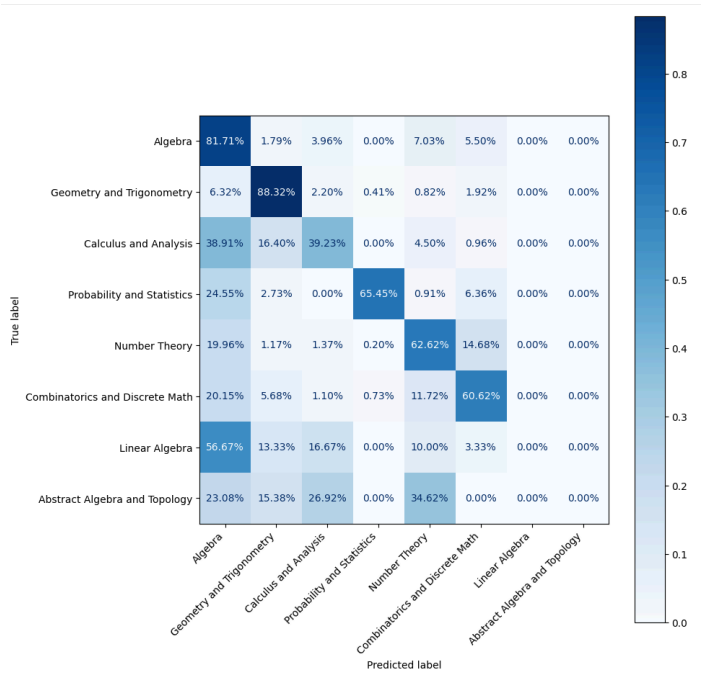

Figure A28. Confusion Matrix, Experiment 8

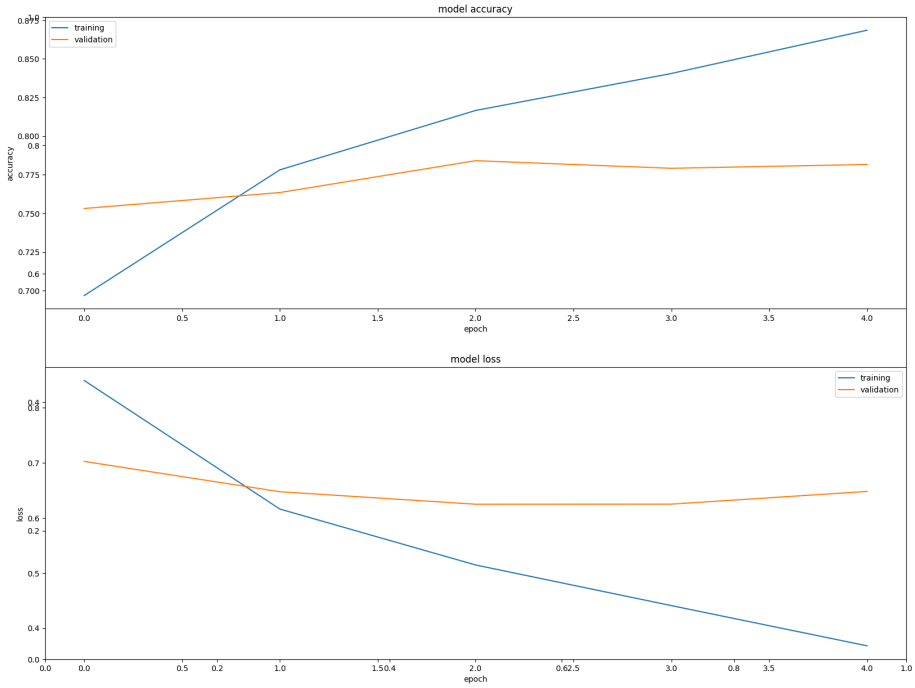

Figure A29. Training and Validation Model Accuracy and Loss, Experiment 9

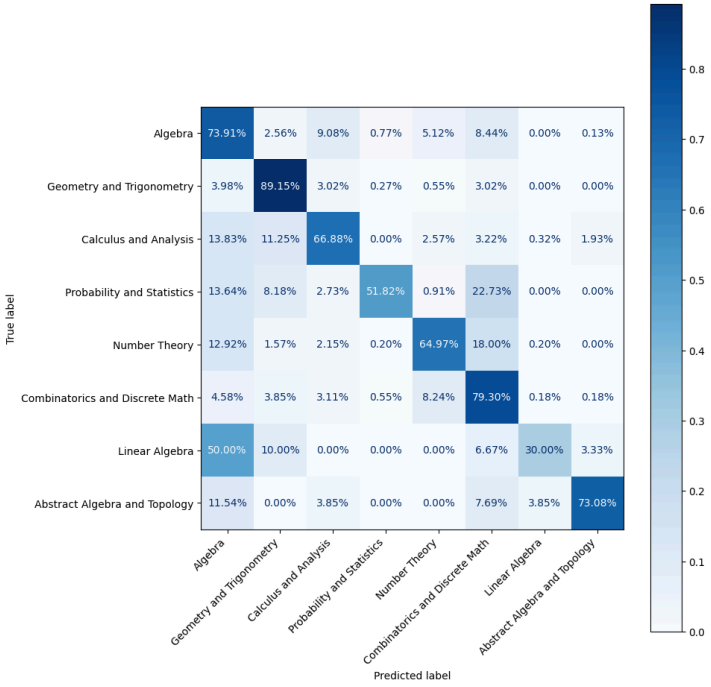

Figure A30. Confusion Matrix, Experiment 9

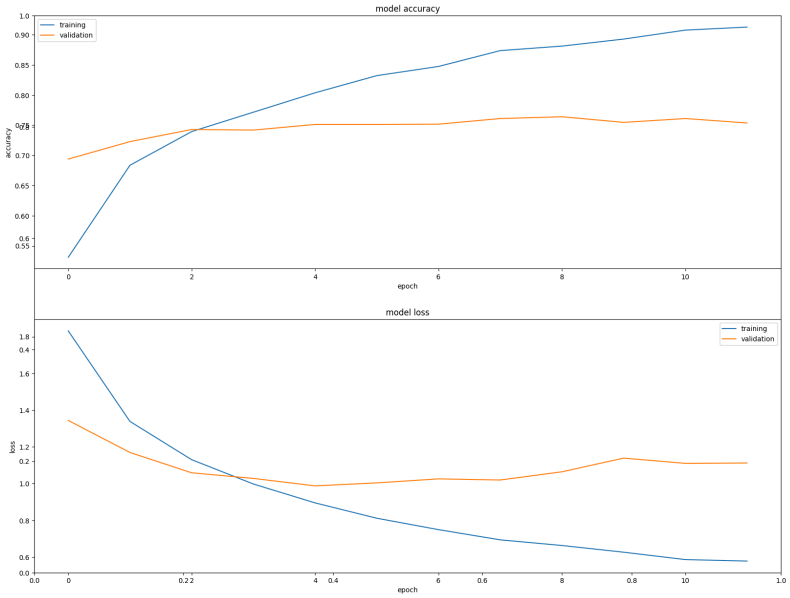

Figure A31. Training and Validation Model Accuracy and Loss, Experiment 10

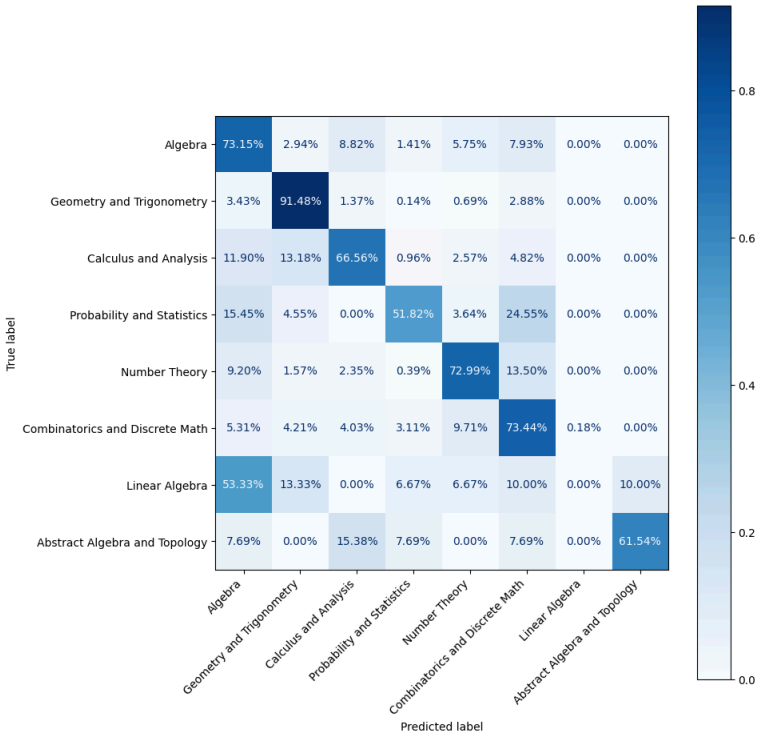

Figure A32. Confusion Matrix, Experiment 10

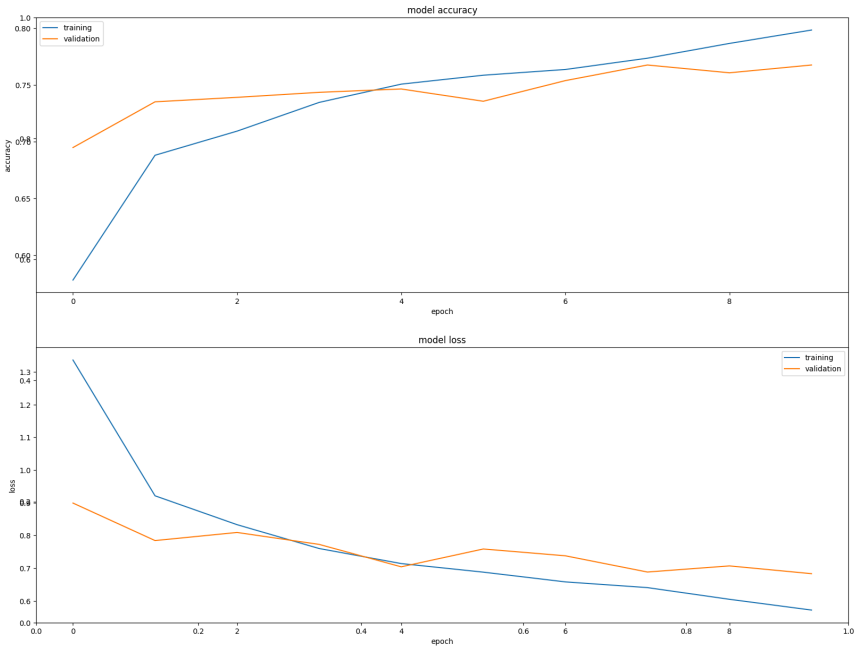

Figure A33. Training and Validation Model Accuracy and Loss, Experiment 11

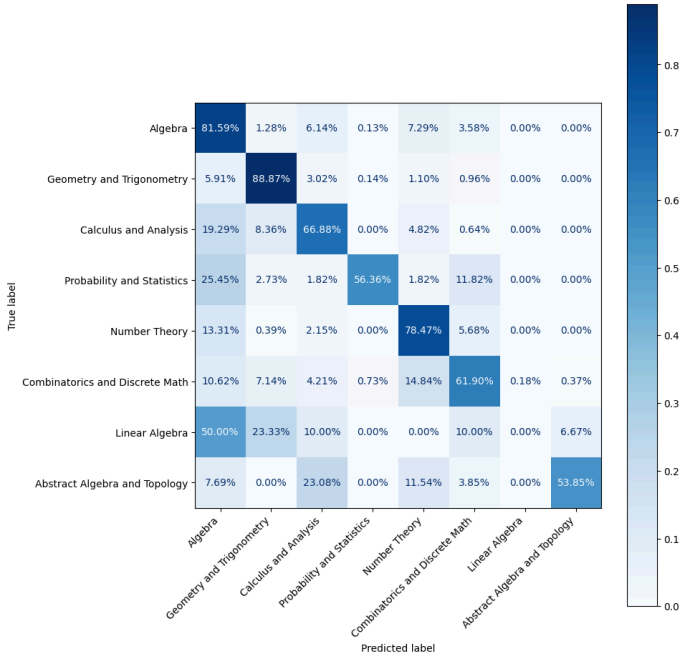

Figure A34. Confusion Matrix, Experiment 11

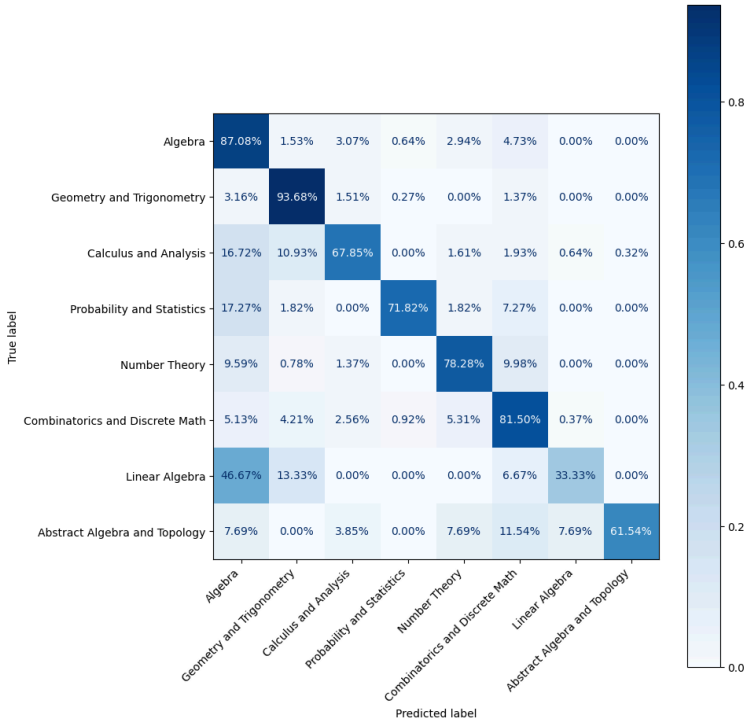

Figure A35. Confusion Matrix, Experiment 12

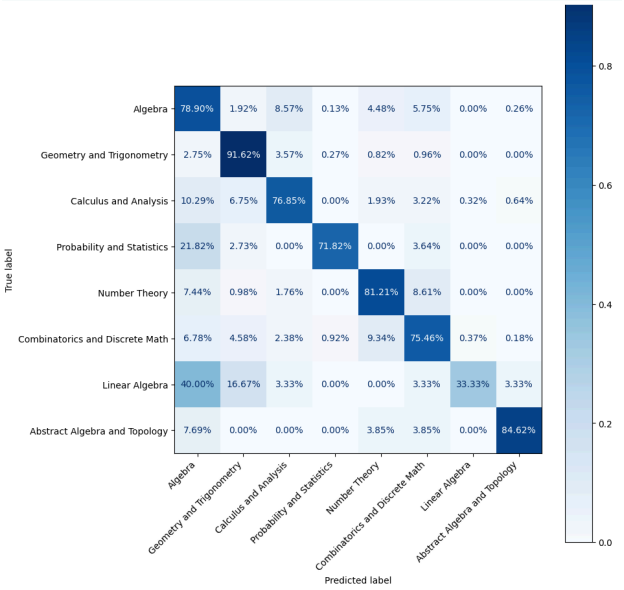

Figure A36. Confusion Matrix, Experiment 13